S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR.

Session-2025-26 (Even)

TAE-I Project Based Learning

AI-Based Voice Assistant (Mini)

Submitted By

Kunal Navdhinge (CS24D006)

AI Technique(s) (2)	Problem Representation (4)	Tools Explored (2)	Outcome (2)	Total (10)
Sign of Student:			**Sign of course In-charge :**	

Industry Problem Context

In the IT industry, **voice assistants** are widely used to simplify human–computer interaction. Instead of manually interacting with computers through keyboards and mice, users can control systems using **voice commands or natural language**.

This technology is used in many applications such as:

- Smart assistants (Alexa, Google Assistant, Siri)
- Voice-controlled automation systems
- Accessibility tools for disabled users
- Smart homes and IoT devices
- Desktop automation tools
-

The main objective is to create a system that can:

- Understand user voice commands
- Process the command using AI/NLP
- Perform system actions automatically

The **ARIA Voice Assistant** solves this problem by allowing users to control their computer using **voice or text commands**, enabling faster and more intuitive interaction.

Feasibility in IT Systems

In real-world IT systems, voice assistants operate using a combination of:

- **Speech Recognition**
- **Natural Language Processing**
- **System Automation**

In the ARIA system:

- **Voice input** is captured using a microphone.
- Speech is converted into **text using Speech Recognition**.
- The command is processed using a **command-processing module**.
- The system performs actions such as opening applications, searching the web, or monitoring system performance.

The system is highly feasible because:

- Python provides libraries for speech recognition and automation.
- System commands can be executed using built-in OS interfaces.
- GUI frameworks like Tkinter allow easy user interaction.

Thus, the project demonstrates a **practical implementation of AI-driven automation in desktop systems**.

AI Techniques Used in Industry

1. Speech Recognition

Speech recognition converts spoken words into text so the system can understand user commands.

Libraries used:

- SpeechRecognition
- Google Speech API
- CMU Sphinx (offline recognition)

This enables the ARIA assistant to capture voice input from the microphone and convert it into text for further processing.

---

2. Natural Language Command Processing

After converting speech to text, the system processes the command using natural language techniques such as keyword matching and fuzzy matching.

Techniques used:

- Keyword matching
- Fuzzy matching (difflib)

Example:

User command:

"open youtube"

3. Text-to-Speech (TTS)

Text-to-Speech converts system responses into spoken audio so the assistant can communicate with the user.

Library used:

- pyttsx3

Example response:

"Opening YouTube."

Representation/Modeling in Industry Systems

In the ARIA system, the voice assistant is modeled using a modular architecture.

Each component performs a specific function.

Main Components

1. Input Layer

Handles voice or text commands.

Modules:

- speech_engine.py
- GUI input field

---

2. Processing Layer

Processes and interprets commands.

Module:

- command_processor.py

Responsibilities:

- Intent recognition
- Command classification
- Action dispatching

---

3. Execution Layer

Performs system actions.

Modules:

- system_controller.py
- system_info.py

Examples:

- Open applications

- Take screenshots
- Shutdown system
- Show CPU usage

---

4. Output Layer

Provides feedback to the user.

Modules:

- tts_engine.py
- GUI command logs

This layered structure makes the system scalable and maintainable.

Tools & Technologies in Industry

The ARIA assistant is developed using modern tools commonly used in software development.

Programming Language

Python

Reason:

- Large AI ecosystem
- Cross-platform compatibility
- Extensive automation libraries

Libraries Used

Library	Purpose
Speech Recognition	Voice to text
PyAudio	Microphone input
pyttsx3	Text-to-speech
psutil	System monitoring
pyautogui	Automation & screenshots
difflib	Fuzzy command matching

GUI Framework

Tkinter

Features:

- Dark themed interface
- Waveform visualization
- System monitoring bars
- Command logs

Python + NetworkX is commonly used for prototyping and proof-of-concept, before deployment in production systems.

Industry-Level Outcome

The ARIA assistant provides several industry-level capabilities.

Automation

Users can perform tasks like:

- Opening websites
- Launching applications
- Taking screenshots
- Controlling system volume

System Monitoring

The assistant can provide:

- CPU usage
- RAM usage
- Battery status
- IP address

Efficiency

Benefits include:

- Faster interaction with the computer
- Reduced manual work
- Hands-free operation

Logging and Tracking

All commands are stored in a log file:

logs/command_history.log

Example:

[2026-03-13 20:19:46] CMD: open calculator

[2026-03-13 20:20:46] CMD: shutdown

Real-World Industry Example/use

Smart Assistants

Examples:

- Amazon Alexa
- Google Assistant
- Apple Siri

These systems perform tasks using voice commands.

Desktop Automation

Developers and IT professionals use voice assistants for:

- Opening development tools
- Running scripts
- Monitoring system performance

Accessibility Systems

Voice assistants help people with disabilities by allowing hands-free control of computers.

Smart Offices

Organizations integrate voice assistants to:

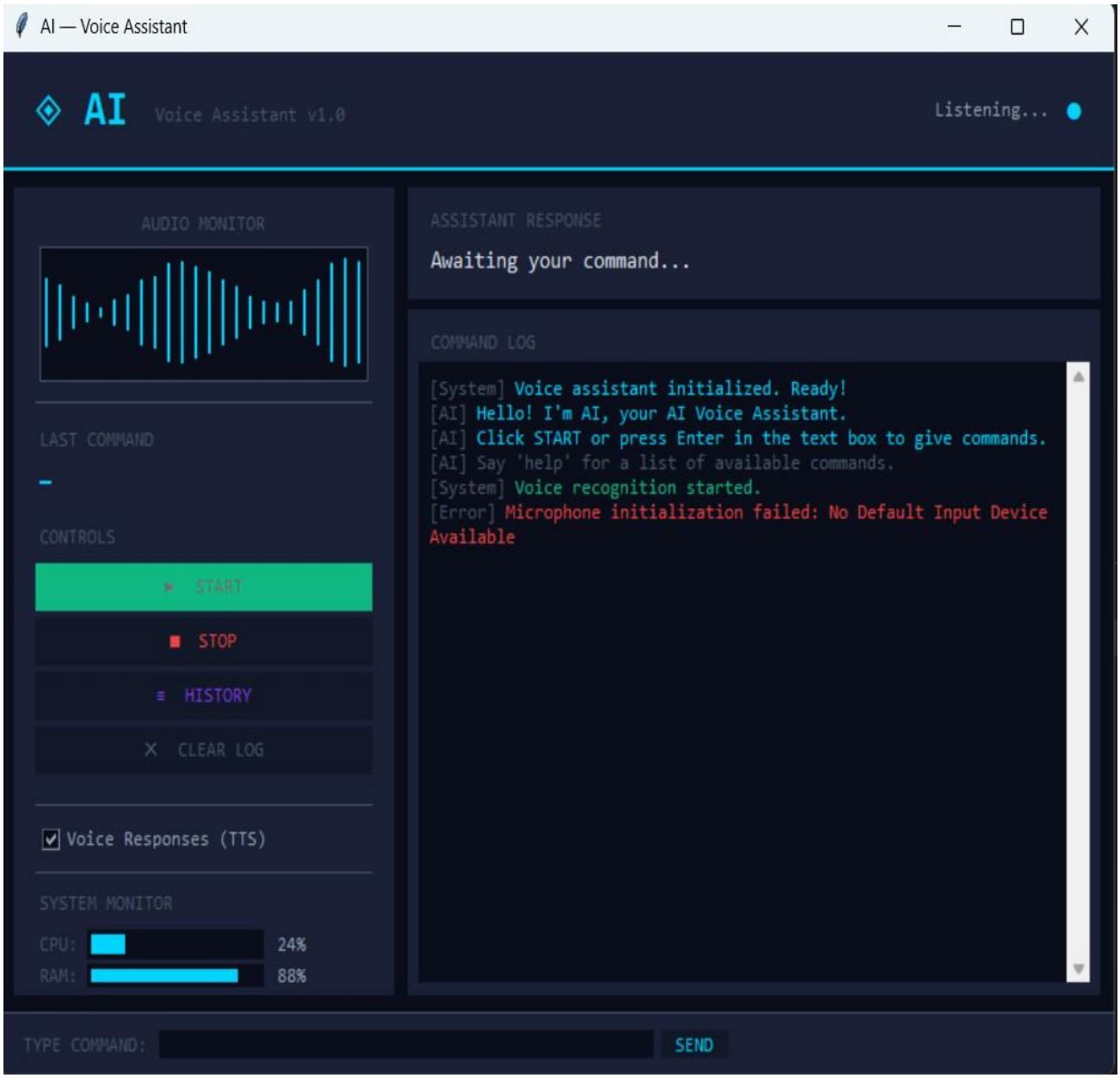
- Control meeting systems
- Manage schedules
- Automate workflows

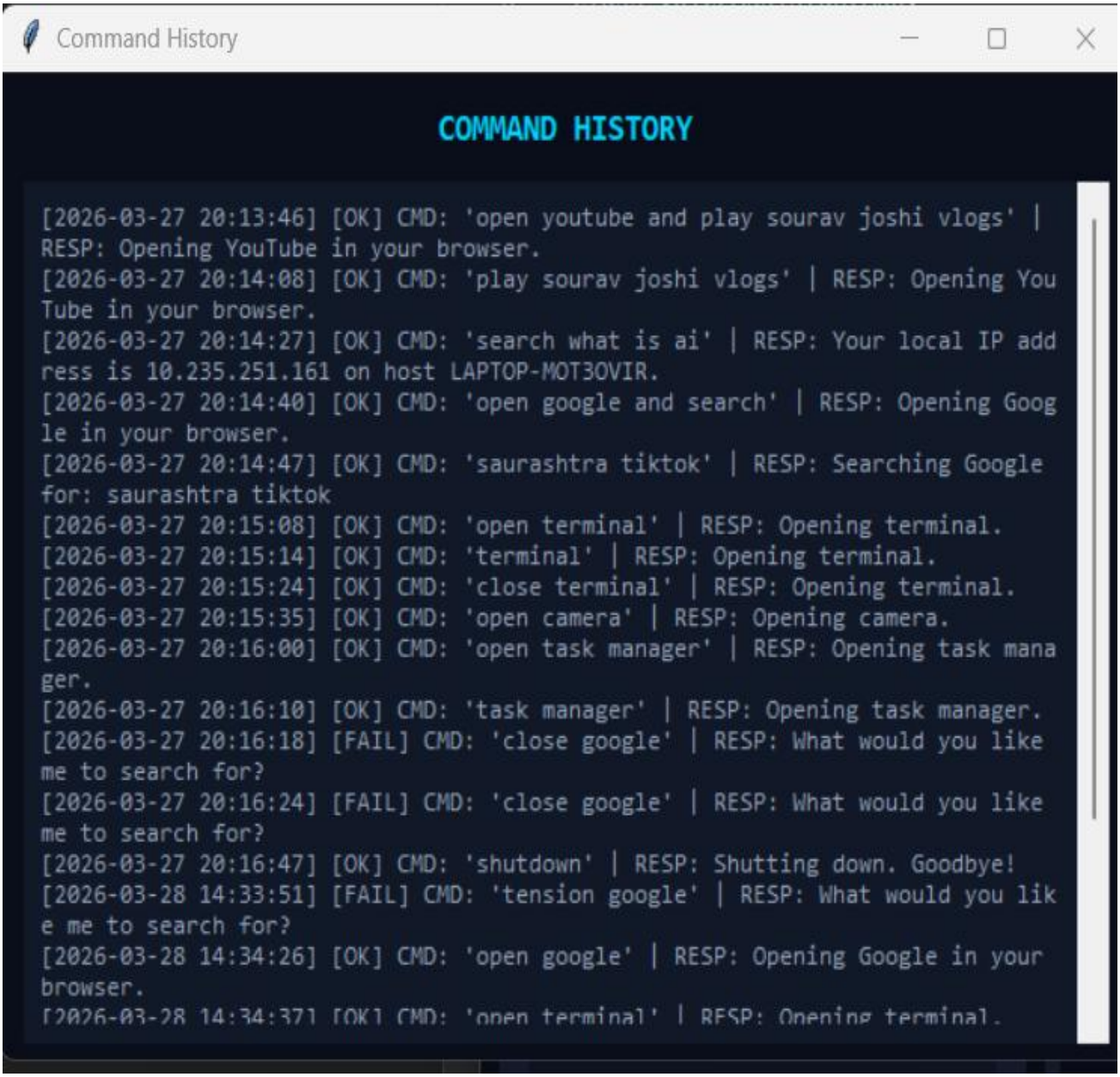
Implementation & Result

This is a **smart AI Voice Assistant** made using Python that can:

- Listen to your voice 🎤
- Understand commands 🤖
- Perform tasks on your laptop 💻
- Give responses using voice 🔊

It also has a **GUI (Graphical User Interface)** built using **Tkinter**.




Command History
COMMAND HISTORY
[2026-03-27 20:13:46] [OK] CMD: 'open youtube and play sourav joshi vlogs' | RESP: Opening YouTube in your browser.
[2026-03-27 20:14:08] [OK] CMD: 'play sourav joshi vlogs' | RESP: Opening YouTube in your browser.
[2026-03-27 20:14:27] [OK] CMD: 'search what is ai' | RESP: Your local IP address is 10.235.251.161 on host LAPTOP-MOT3OVIR.
[2026-03-27 20:14:40] [OK] CMD: 'open google and search' | RESP: Opening Google in your browser.
[2026-03-27 20:14:47] [OK] CMD: 'saurashtra tiktok' | RESP: Searching Google for: saurashtra tiktok
[2026-03-27 20:15:08] [OK] CMD: 'open terminal' | RESP: Opening terminal.
[2026-03-27 20:15:14] [OK] CMD: 'terminal' | RESP: Opening terminal.
[2026-03-27 20:15:24] [OK] CMD: 'close terminal' | RESP: Opening terminal.
[2026-03-27 20:15:35] [OK] CMD: 'open camera' | RESP: Opening camera.
[2026-03-27 20:16:00] [OK] CMD: 'open task manager' | RESP: Opening task manager.
[2026-03-27 20:16:10] [OK] CMD: 'task manager' | RESP: Opening task manager.
[2026-03-27 20:16:18] [FAIL] CMD: 'close google' | RESP: What would you like me to search for?
[2026-03-27 20:16:24] [FAIL] CMD: 'close google' | RESP: What would you like me to search for?
[2026-03-27 20:16:47] [OK] CMD: 'shutdown' | RESP: Shutting down. Goodbye!
[2026-03-28 14:33:51] [FAIL] CMD: 'tension google' | RESP: What would you like me to search for?
[2026-03-28 14:34:26] [OK] CMD: 'open google' | RESP: Opening Google in your browser.
[2026-03-28 14:34:37] [OK] CMD: 'open terminal' | RESP: Opening terminal.

References

- **Tkinter**: Standard GUI library
  - Official Docs: https://docs.python.org/3/library/tkinter.html
- **speech_recognition**: Speech-to-text using Google Web Speech API
  - PyPI: https://pypi.org/project/SpeechRecognition/
  - GitHub: https://github.com/Uberi/speech_recognition
  - Setup: pip install SpeechRecognition pyaudio
- **psutil**: System and process utilities (CPU, memory, processes)
  - Docs: https://psutil.readthedocs.io/en/latest/
  - PyPI: https://pypi.org/project/psutil/
- **pyttsx3**: Offline text-to-speech engine (mentioned in related files)
  - Docs: https://pyttsx3.readthedocs.io/

Key External URLs Hardcoded in Code (command_processor.py)

https://www.youtube.com (open_youtube)
https://www.google.com (open_google)
https://www.github.com (open_github)
https://www.wikipedia.org (open_wikipedia)
https://mail.google.com (open_gmail)
https://maps.google.com (open_maps)
https://www.google.com/search?q={query} (search_google)